

Esercitazione 1

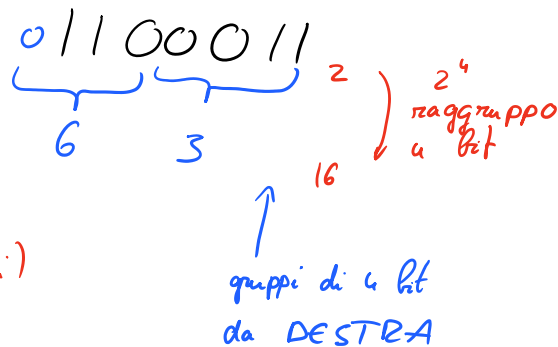
venerdì 24 febbraio 2017 10:45

① $99_{10} \rightarrow 99_2 \rightarrow 99_{16}$

base $m \Rightarrow m$ valori diversi
 $0 \dots m-1$ cifre

99	2	1
49	2	1
24	2	0
12	2	0
6	2	0
3	2	1
1	2	1

0x
segnala
che sto
usando Hex
(in tastiera
non ho pedici)



② $0x52 = 0101\ 0010_2$

$0x1010101 = 0001\ 0000\ 0001\ 0000\ 0001\ 0000\ 0001_2$

$0xF3B4C = 1111\ 0011\ 1011\ 0100\ 1100_2$

③ $1100\ 1110\ 1110\ 011_2 =$

6 3 5 6 3 ⑧ = 2^e gruppetti di 3 da dx

= $0x\ 6\ 7\ 7\ 3$

④ MODULO E SEGNO: (8 bit)

$-20 =$ 1 | 00 10100 MS = se non ci fosse
MS sarebbe 128_{10}

20	2	0
10	2	0
5	2	1
2	2	0
1	2	1

-114

1 111 001 0

114	2	0	Δ
57	2	1	
28	2	0	
14	2	0	
7	2	1	
3	2	1	
1	2	1	

vantaggi:

- numeri positivi uguali

svantaggi:

- perdita di valori positivi scrivibili

- due zeri (+0 e -0)

- non riesco a fare aritmetica con valori molto alti o bassi (overflow)

⑤ COMPL. A 2

: 8 bit, 256 caselle $[-128, +127]$

① DEVO SAPERE:

QUANTI
BIT
USARE

numeri positivi uguali
(primo bit spento)

numeri negativi sono
complementati

① non ho 2 zeri

② c'è una discontinuità

grave! $\rightarrow$ 0 111 111 1 127
 $\rightarrow$ 1 0000000 -128

come? ① binario

② compl a 1

③ aggiungi 1

① dimentico il meno

X20 : 00010100₂

(su 8 bit!)

... 0 1 1 1 1 1 1 ② compl a 1

$$\begin{array}{r}
 11101011 \downarrow + \textcircled{2} \text{ compl. a } 1 \\
 \quad \quad \quad 1 = \textcircled{3} \text{ aggiungo } 1 \\
 \hline
 11101100_{c2}
 \end{array}$$

Metodo 2: $00010100_2 = 20_{10}$

$$11101100_{c2}$$

↑
ho trovato
il primo 1, da ora in poi invertito

⑥ $01001 <?> 10001$

ms: $+9 > -1$

c2: $10111_2 > 01111_2$

$$101 \quad 11101$$

ms: $-1 > -13$

c2: $011 = 00011$

$$\begin{array}{r}
 101- \\
 \underline{1} = \\
 100 \\
 01
 \end{array}$$

⑦ eccesso 128:

$$000000000 = -128_{10}$$

$$\text{-----} = -128_{10}$$

$$\text{1111111} \quad \text{---} \quad +127_{10}$$

vantaggi: no salto a metà
svantaggi: positivi sbalzati

$$-20_{10} = 01101100$$

$$-20 + 128 = 108$$

ec. 128

←
aggiungo 128
(va da -128 a +127)

108	2	0
54	2	0
27	2	1
13	2	1
6	2	0
3	2	1
1	2	1

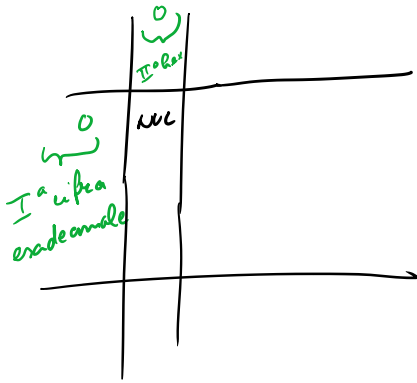
$$00000000_{ec. 128} = -128_{10}$$

$$0 - 128 = -128$$

$$00011100 = -100_{10}$$

$$28 - 128 = -100$$

⑧ ASCII (su 7 bit)



① 7 bit sono pochi.

es: $NUC = \underbrace{0000}_{I^a \text{ cifra}} \underbrace{0000}_{II^a \text{ cifra}}$
 $= 0x00$

$\Rightarrow$ Extended ASCII (8 bit)

UNICODE (16 bit)

$$Bit = 0x42 \quad 69 \quad 74$$

$$2012 = 0x32 \quad 30 \quad 31 \quad 32$$

⑨ $13,25 =$

$1101 \quad 01$

$1101,01 \Rightarrow$

forma normalizzata

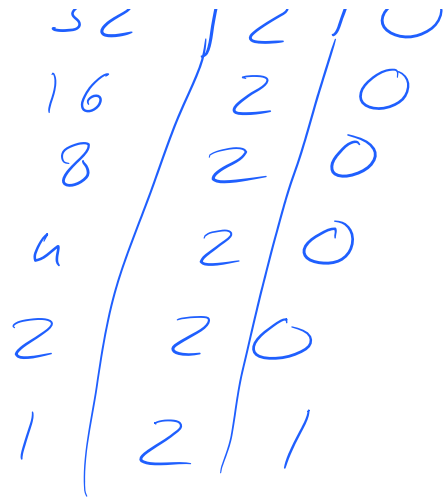
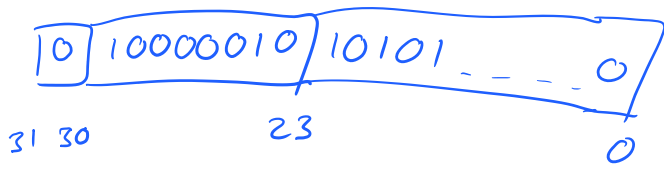
$1,10101 \cdot 2^3$

$0,25 \mid 2 \mid 1$
 $12 \mid 2 \mid 0$

$3 + 127 = 130$

$101000001010101 \dots 0$

$130 \mid 2 \mid 0$
 $65 \mid 2 \mid 1$
 $32 \mid 2 \mid 0$
 $16 \mid 2 \mid 0$



$$AFA_{16} = 10$$

$$\begin{array}{c} \text{AFA} \\ \hline 1010 \quad 1111 \quad 1010 \end{array}$$

$$\begin{array}{c|c|c} AFA & A & O \\ \hline AF & A & F \\ A & A & O \\ 1 & A & 1 \end{array}$$

$$\begin{array}{c|c} \widehat{AFA} & A \\ \hline & \end{array}$$

$$\widehat{AFA} : A = 11$$

$$SA : A =$$

Esercizio 1

venerdì 10 marzo 2017 10:45



① $\text{not}(\text{not } A \text{ or } B) \text{ and } C$

$\neg(\neg A \vee B) \wedge C$

② $(\bar{A} \text{ or } B) \text{ and } (C \text{ or } D)$

$\neg(\neg A \vee B) \wedge (C \vee D)$

③

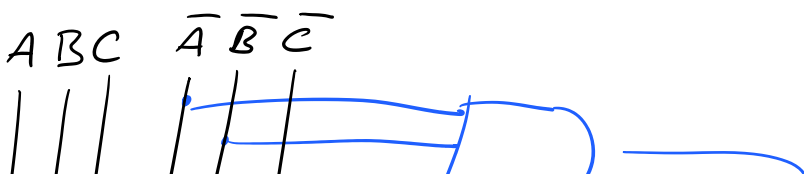
A	B	$A \wedge (\neg B)$	$(\neg A) \wedge B$	$A \wedge B$	OR
0	0	F	F	F	F
0	1	F	T	F	T
1	0	T	F	F	T
1	1	F	F	T	T

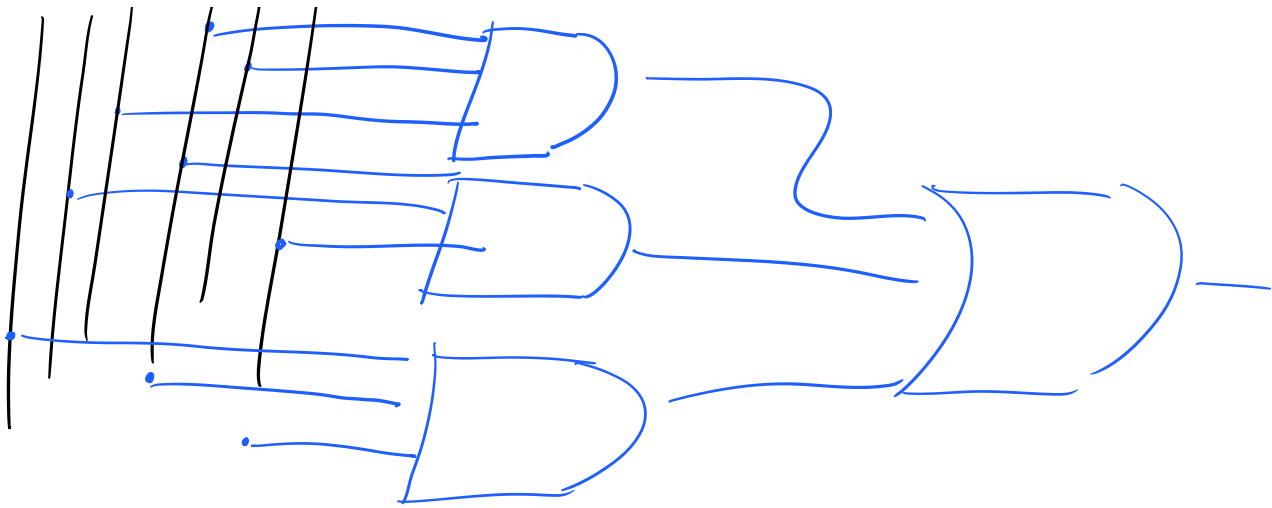
④

A	B	C	$\bar{A}BC$	$A\bar{B}\bar{C}$	$A\bar{B}C$	$A\bar{B}\bar{C}$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	1	0	0
1	0	1	0	0	1	0
1	1	0	0	0	0	1
1	1	1	0	0	0	0

⑤ con la tabella di verità posso rappresentare più funzioni.
(→ u)

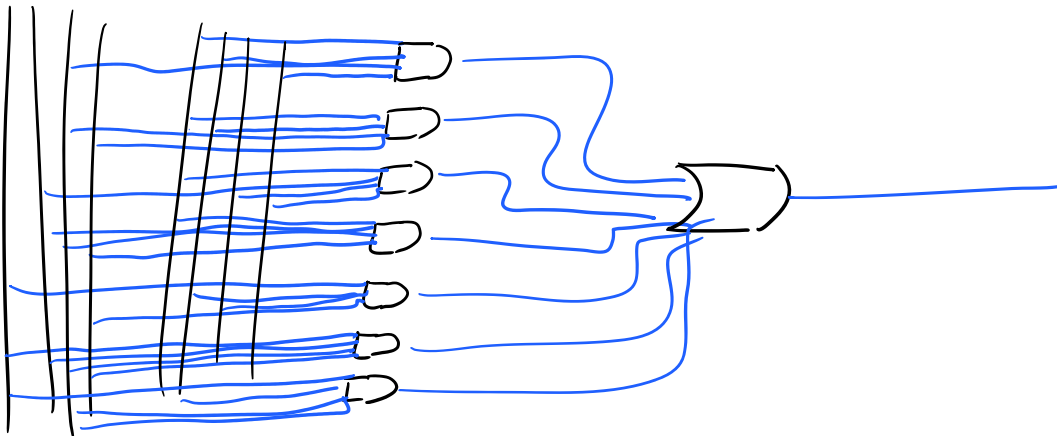
⑥ guarda dove è vero (è la somma delle combinazioni vere (prodotti))
 $\bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C}$





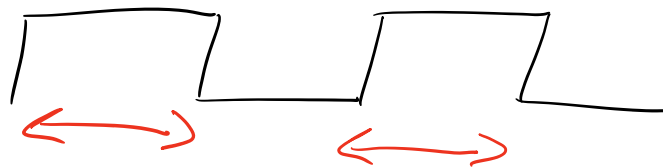
⑦ $\overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}CD + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D$

ABCD $\overline{A}\overline{B}\overline{C}\overline{D}$



⑭ resp. 4

① clock :

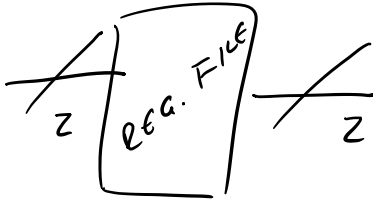


onda
quadrata
con
periodo
costante

③ $f = \frac{1}{T} = \frac{1}{0,02 \cdot 10^{-6}} = 50 \cdot 10^6 \text{ Hz}$
 $= 50 \text{ MHz}$

②

4)



x leggere ho bisogno di 2 ingressi x endowere iniziali
e di 2 uscite

x scrivere: dove scrivo e cosa scrivo (2 entrate)

④ 1 ingresso x scrittura/lettura

=> 5 ingressi e 2 uscite

⑤ 4096 Kbit
word a 32 bit

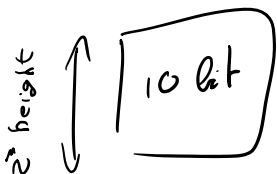
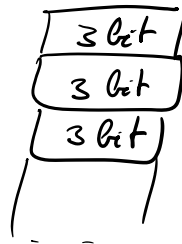
$$4096 = 2^{12}$$

$$32 = 2^5$$

$$\frac{2^{12} \cdot \cancel{10^3}^{2^{10}}}{2^5} = 2^{17}$$

⑥ 10 bit
3 bit

$$= 4$$



3 bit ind.

2^3 diverse locazioni

max 8 locazioni

grandezza: $\frac{\text{dim. tot.}}{\text{altezza}}$

⑦ 30 bit addr] ① ^{2³⁰ indirizzi}
 64 bit word NO ^{30 indirizzi}

$$2^{30} \cdot 64$$

$$2^{30} \cdot 2^6$$

$$= 2^{36} \text{ bit} : 2^3$$

$$= 2^{33} \text{ B} : 2^{20} =$$

$$2^{13} \text{ MB}$$

⑨ SRAM < DRAM
 DRAM 5-10 volte più lente

⑩ 0x FFFFFFFFA NO!!

60
 80
 A0
 C0
 E0

$\begin{array}{r} \textcircled{10} \\ 11- \\ 10- \\ \hline 1 \end{array}$	$\begin{array}{r} \textcircled{HEX} \\ B- \\ A= \\ \hline 1 \end{array}$
	$\begin{array}{r} 3 \\ A+ \\ 7= \\ \hline 11- \\ 7= \\ \hline A \end{array}$

$$1 + F = 10$$

$$9 + 1 = 10$$

$$9 + 2 = 11$$

$$X = (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot \bar{C}) + (A \cdot B \cdot C)$$

$$Y = (A \cdot B \cdot C)$$

00		abcdeF
01		bc
10		abged
11		abcdg

		a	b	c	d	e	F	g
00		1	1	1	1	1	1	0
01		0	1	1	0	0	0	0
10		1	1	0	1	1	0	1
11		1	1	1	1	0	0	1

Esercitazione 2

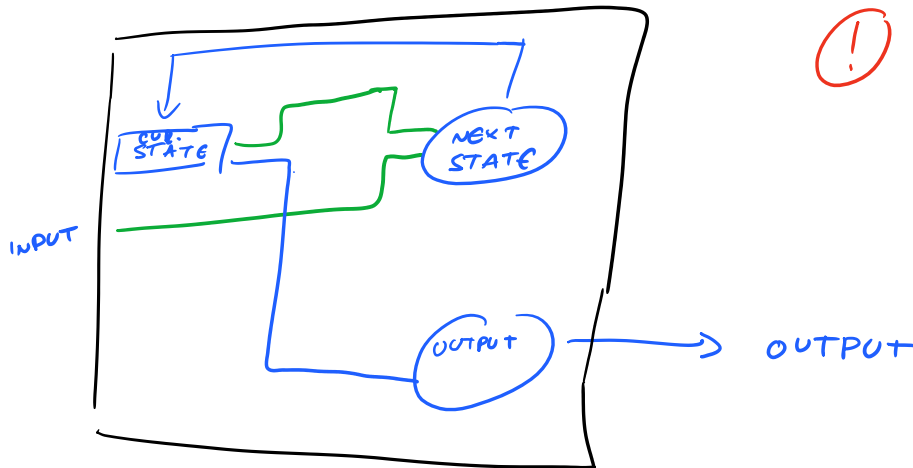
venerdì 17 marzo 2017 10:47

→ FSM (Finite State Machine)

MOORE MACHINE
(output dipende solo dallo stato)

↑ usata da noi

MEALY MACHINE
(output = stato corrente + input)

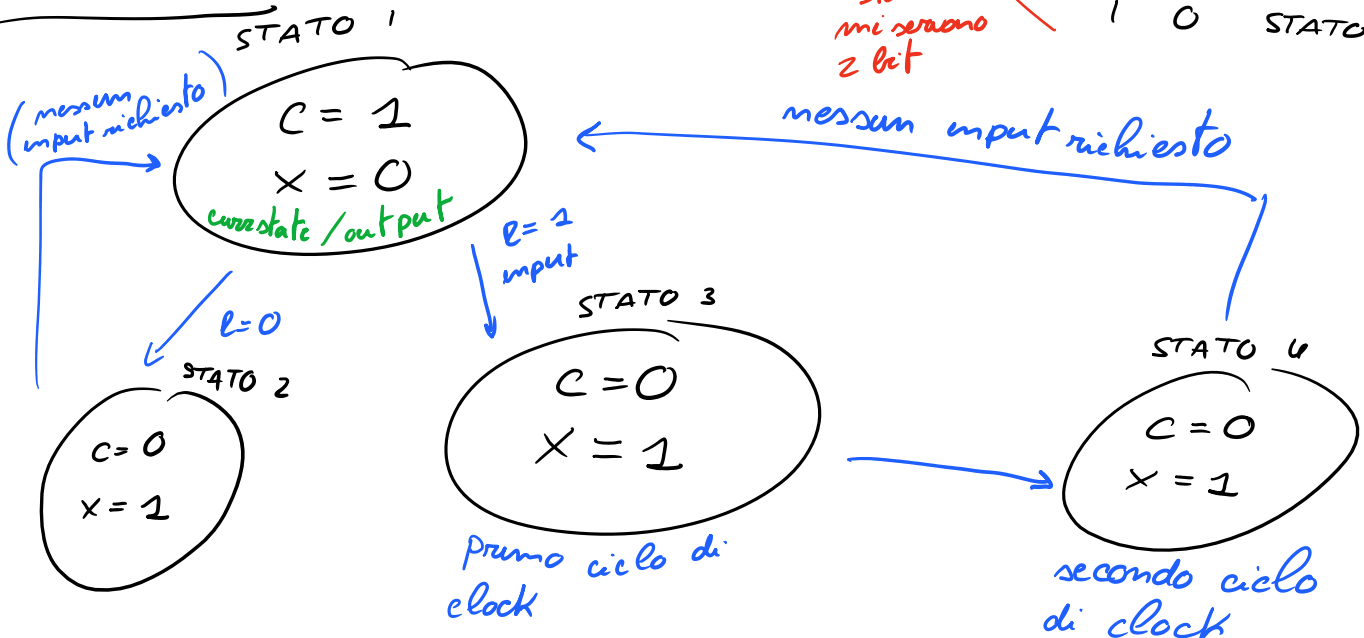


⚠ NB: l'input non determina direttamente l'output

Esercizio 1:

visto che ho 4 stati mi servono 2 bit

X	C	
0	1	STATO 1
1	0	STATO 2



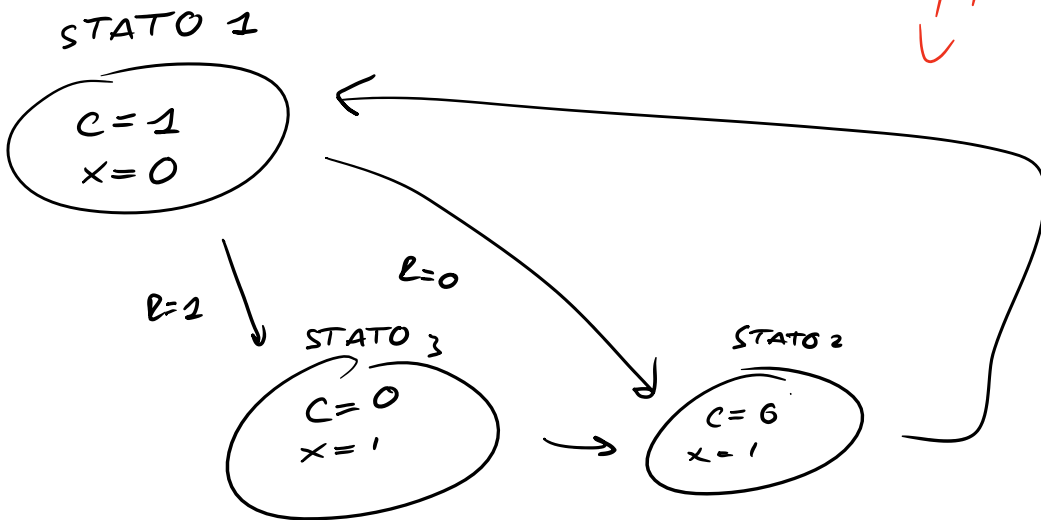
STATO 3 ≠ STATO 2 e STATO 4
in quanto stessa destinazione e stesso output (STATO 1)

STATO 1

OTTIMIZZAZIONE

STATO 3 simile e STATO 4
in quanto stessa destinazione e stesso output
(STATO 1)

OTTIMIZZAZIONE



① NON ha comunque un risparmio di memoria (per 3 o 4 stati: sempre 2 bit)
MA risparmio porte logiche (e quindi più veloce)

ESERCIZIO 11

I° cifra hex II° cifra hex

0010 0010 1110 1011 0000 0000 0110 0000
 08 rs rf 16 bit imm
 (binario) (binario) (binario)

addi \$11 \$23 96
 rf rs imm

sub <dest>, <operando> <operando>

lw \$t0, 56(\$s0)
 ← un byte (offset)

↑ registro con indirizzo di partenza

add \$t1, \$s1, \$t0

sw \$t1, 68(\$s0)

ESERCIZIO 4:

addi \$5, \$6, 100

↑_x sommare una costante

ESERCIZIO 7:

slt \$t0, \$t0, \$t8
UUU UUW UUUU UU
73 60 74 20 24 31 20 24 31 20 26 31

ESERCIZIO 5

add \$t0, \$s0, \$s2 $t_0 \leftarrow 15$

add \$t1, \$s2, \$s3 $t_1 \leftarrow 120$

sub \$t2, \$t1, \$t0 $t_2 \leftarrow 120 - 15 = 105$

ESERCIZIO 8:

① sub \$t0, \$t0, \$t0
xor \$t0, \$t0, \$t0

② add \$t1, \$t0, \$t0 ← mi servono
add \$t1, \$t0, \$t0 ← 2 esercizi

③ add \$t0, \$t0, \$t0 ← $4 = 2^2 = 2 \cdot 2 = 2 \cdot (1 \cdot 2)$

add \$t0, \$t0, \$t0

- (' c)

ESERCITAZIONE 3

venerdì 24 marzo 2017 10:45

- CODICE (text) va in TEXT SEGMENT (sotto "user text segment")
- SETTINGS: disabilita "load exception handler"
- ISA le istruzioni vere e proprie (del MIPS)

ASSEMBLATORE
fa ottimizzazione

ASM: il mio codice

la \$t7, valori ← è ovvio che è una pseudoinstruzione in quanto solo "valori" occupa 32 bit

lui
ori
ISA

⊕ la
ASM

bgt \$t0, \$zero
blt \$t0, \$zero
ASM

slt
beg
ISA

ESERCIZIO 41

se i byte non
sono utilizzati
tutti vengono riempiti
da 0 dopo

- data 0x10010000
parole a 32 bit in C2
 - word
 - ascii
 - asciiZ
 - space 2
- indirizzo di partenza
- # stringa null terminated (contiene 0)
- # mette tutti 0 in 2 byte

eol: sta in /n

ogni byte è indirizzabile

ESECRITAZIONE 4

venerdì 31 marzo 2017 10:48

→ x usare procedure mi servono CONVENZIONI!

PRIMA: procedura

exit

DOPO: chiamante 1

CONVENZIONE: parametro in ingresso

procedura

ritorno

CONVENZIONE: ritorno in uscita

→ CODICE DI AWIO:
chiama "main"
e alla fine ritorna EXIT

→ NOP: usata x riempire RAM con istr. vuote

ESECRIZIO 1

.data 0x10010000

numeri: .word 1,2,3

.text

.globl main

main:

lw \$t0, numeri

la \$t1, numeri

lw \$t2, 8(\$t1)

dato che numeri è la prima istruzione in .data, allora 1 è alla posizione 0x10010000

se ho un 'array' carico il primo elemento con lui

\$t0 = 1

\$t1 = 0x10010000

\$t2 = 3 cont. decimale

ESECRIZIO 3:

KNOW 100 (il valore iniziale) se la procedura ha il suo

N.B.: \$50 vale 100 (il valore iniziale) se la procedura fa il suo dovere (cioè resetta \$50 al valore iniziale) e viene cioè preservato in chiamata

Esercizio 2:

exit → ritorno senza valore restituito
exit 2 → valore restituito

A) RAPPRESENTAZIONE INFORMAZIONE

① $99_{10} = 1100011_2 = 143_8 = 63_{16}$
 (= 0x63)

99	2	1
49	2	1
24	2	0
12	2	0
6	2	0
3	2	1
1	2	1

001100011
 1 4 3 8 (prendo gruppi di 3 bit partendo da dx)

01100011
 6 3 16 (gruppi 4 bit ..)

④ $011001110110011_2 = 63563_8 = 6773_{16}$

6 7 7 3
 011001110110011
 6 3 5 6 3

$0x71A9_{16} = 070651_8 = 0111000110101001_2$

↑ si potrebbe omettere

000111000110101001
 0 7 0 6 5 1

$736_8 = 1DE_{16} = 11101110_2$

736
 000111011110
 1 13=D 14=E

⑤ M&S (8 bit: primo bit segno, altri bit valore assoluto)

- $20_{10} = 10010100_{MS}$

- $114_{10} = 11110010_{MS}$

- $59_{10} = 10111011_{MS}$

(+) $33_{10} = 00100001_{MS}$

$10_{10} = 00001010_{MS}$

114	2	0
57	2	1
28	2	0
14	2	0
7	2	1
3	2	1
1	2	1

54	2	1
27	2	1
14	2	0
7	2	1
2	2	1

$$\begin{array}{r|l|l} 14 & 2 & 0 \\ 7 & 2 & 1 \\ 3 & 2 & 1 \\ 1 & 2 & 1 \end{array}$$

⑥ CAz (se positivo: primo bit 0, altri bit normali.
se negativo: ottengo positivo, invertito bit, sommo 1)
N.B.: su 8 bit!

$$-20_{10} = 11101100$$

$$\left(\begin{array}{l} +20_{10} = 00010100_2 \text{ mv} \\ 11101100_2 \text{ (o meglio compl. a 1, e1) +} \\ \hline 12 \\ \hline 11101100 = -20_{10} \end{array} \right) =$$

$$-114_{10} = 10001110$$

$$\left(\begin{array}{l} +114_{10} = 01110010 \text{ mv} \\ 10001101 \text{ e1 +} \\ \hline 12 \\ \hline 10001110 \end{array} \right) =$$

$$-59$$

$$\left(\begin{array}{l} +59 : 00111011 \\ 11000101 \end{array} \right)$$

METODO 2: ① numero uguale fino al primo 1 partendo da dx

② dal successivo scambio

$$\begin{array}{r} \textcircled{8} \quad \begin{array}{r} \overset{1}{7} \overset{1}{8} 9_{16} + \\ 987_{16} = \\ \hline 1110_{16} \end{array} \end{array}$$

$$9+7=16 = 10_{16} \text{ (NON A, } 1 \cdot 16^1 + 0 \cdot 16^0)$$

$$8+8+1=17 = 11_{16}$$

$$\textcircled{9} \quad 432_{16} = 3212_5$$

$$12_5 = 7_{10}$$

$$\begin{array}{r|l|l} 43^2_2 & 5 & 2 \\ 8^2_6 & 5 & 1 \end{array}$$

$$1 \cdot 5^1 + 2 \cdot 5^0 = 5 + 2 = 7$$

$$\begin{array}{r|l|l} 4 & 3^2 & 5 \\ 8 & 2^6 & 5 \\ 17 & 5 & 2 \\ 3 & 5 & 3 \end{array}$$

$$1 \cdot 5^1 + 2 \cdot 5^0 = 5 + 2 = 7$$

$$534_5 = 144_{10}$$

$$5 \cdot 5^2 + 3 \cdot 5^1 + 4 \cdot 5^0 = 125 + 15 + 4 = 144$$

⑩ Eccesso 128: (aggiungo 128 al numero dato)

$$-20_{10} = 01101100$$

$$128 - 20 = 108_{10}$$

$$\begin{array}{r|l|l} 108 & 2 & 0 \\ 54 & 2 & 0 \\ 27 & 2 & 1 \\ 13 & 2 & 1 \\ 6 & 2 & 0 \\ 3 & 2 & 1 \\ 1 & 2 & 1 \end{array}$$

$$20_{10} = 10010100$$

$$128 + 20 = 148$$

⑫ $00000000_2 = -128_{10}$

$$? + 128 = 0 \rightarrow ? = -128$$

$$00011100_2 = -100_{10}$$

$$? + 128 = 28 \rightarrow ? = -100$$

⑬ disco 2 GB foto 3 MB

$$2 \text{ GB} = 2 \cdot 2^{30} \text{ B}$$

$$3 \text{ MB} = 3 \cdot 2^{20} \text{ B}$$

$$\frac{2 \cdot 2^{30}}{3 \cdot 2^{20}} = 0,66 \cdot 2^{10} \sim 675$$

⑥ 13,25

$$\begin{array}{r|l} 13 & 2 \\ 6 & 2 \\ 3 & 2 \\ 1 & 2 \end{array} \quad \begin{array}{l} 1 \\ 0 \\ 1 \\ 1 \end{array}$$

$$\begin{array}{r} \boxed{0,25} \times 2 = 0 \\ \hline \boxed{0,50} \times 2 = 0 \\ \hline \boxed{1,00} \end{array} \quad \begin{array}{l} 0 \\ 0 \\ 1 \end{array}$$

$$13 = 1101$$

$$13,25 = 1101.001 = \text{forma normalizzata} = 1. \underbrace{101001}_{\text{mantissa}} \cdot 2^3$$

S	exp (ecc. 127)	mantissa (= parte decimale)
0	10000010	101001 - - -
1 bit	8 bit	23 bit

fp 32

$$3 + 127 = 130_{10} = 128 + 2 = 10000010 = \text{exp ecc. 127}$$

⑦ exp (ecc. 127) mantissa (parte dec)

1	10000000	110 - - - 0
1 bit	8 bit	23 bit

so che: ① segno negativo

② exp. ecc. 127 = 128 $\rightarrow 128 - 127 = 1$ vero exp.

③ forma normalizzata $1,11 \cdot 2^1 \rightarrow$ numero vero: 11,1

$\rightarrow$ numero: $-3,5$

$$\begin{array}{cc} \underline{3} & \underline{5} \\ \downarrow & \downarrow \\ 11_2 & 0,1_2 \end{array}$$

① FP
ecc. 127
128

② RETI COMBINATORIE

① $(\overline{A} \text{ OR } B) \text{ AND } C$

② $\text{NOT}(\text{NOT}(\text{NOT}(A) \text{ OR } B) \text{ AND } (C \text{ OR } D))$

③

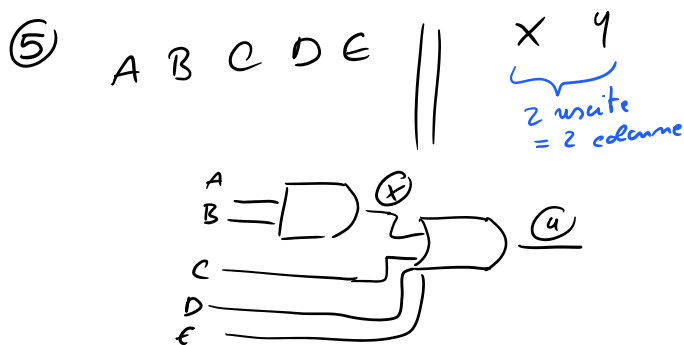
A	B	out
0	0	0
0	1	1
1	0	1
1	1	1

$(A \text{ AND } \overline{B}) \text{ OR } (B \text{ AND } \overline{A}) \text{ OR } (A \text{ AND } B)$

④

A	B	C	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$\overline{A} \text{ AND } B \text{ AND } C$
 OR
 $A \text{ AND } \overline{B} \text{ AND } C$
 OR
 $A \text{ AND } B \text{ AND } \overline{C}$
 OR
 $A \text{ AND } B \text{ AND } C$

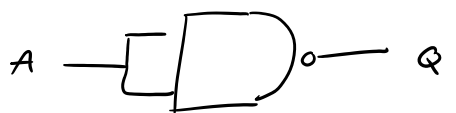


⑥ UNIVERSALITÀ NAND:

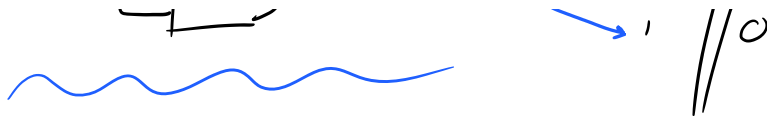
$A \text{ AND } B = \text{AND gate} \rightarrow Q$

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

NOT:



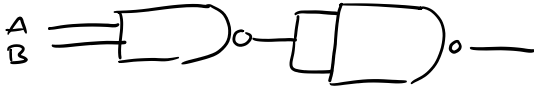
A	Q
0	1
1	0



AND:

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

è l'opposto della
 NAND: **NAND + NOT**



OR:

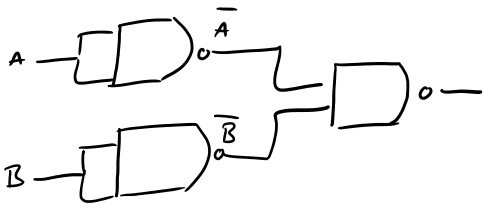
⚠️ uso DE MORGAN:
NAND

$$\text{NOT}(A \text{ AND } B) = \text{NOT}(A) \text{ OR } \text{NOT}(B)$$

$$\overline{A \cap B} = \overline{A} \cup \overline{B}$$

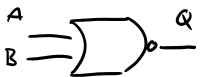
→ voglio $A \cup B$, quindi nego un'altra
 volta A e un'altra B

$$\overline{\overline{A}} \cup \overline{\overline{B}} = A \cup B$$



(aggiungendo un not alla
 fine otterremo un OR)

UNIVERSALITÀ NOR:



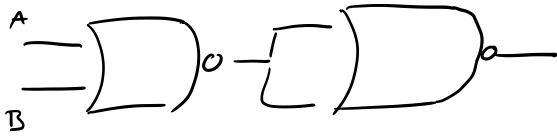
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

NOT



A	Q
0	1
1	0

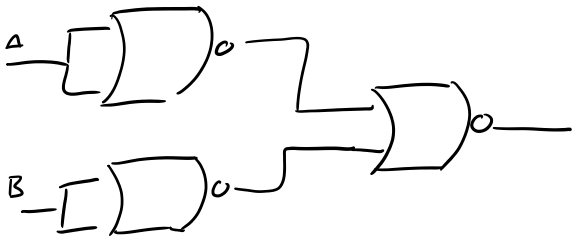
OR = NOR + NOT



AND: uso DE MORGAN

$$\overline{\overline{A} \cup \overline{B}} = \overline{\overline{A}} \cap \overline{\overline{B}}$$

$$\overline{\overline{A}} \cap \overline{\overline{B}} = A \cap B$$



(aggiungendo un not alla fine otterrei NAND)

⑦

A	B	C	D
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

① avrà 1 solo quando una (o più) delle condizioni iniziali saranno vere

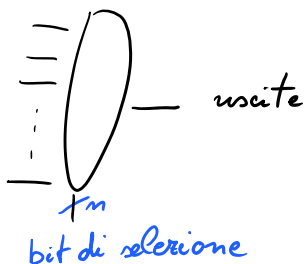
② le condizioni che portano a 1 sono 3: 0 AND 0 AND 1, 0 AND 1 AND 0, 1 AND 0 AND 0

$$\Rightarrow (\overline{A} \cdot \overline{B} \cdot C) + (\overline{A} \cdot B \cdot \overline{C}) + (A \cdot \overline{B} \cdot \overline{C})$$

⑨ CIRCUITI IMPORTANTI:

① MUX:

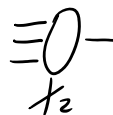
2^m entrate



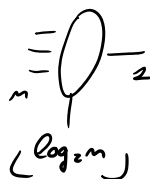
bit di selezione



2 entrate (A0 A1)
1 bit di selezione
(0 → passa A0)
(1 → passa A1)
1 bit uscita



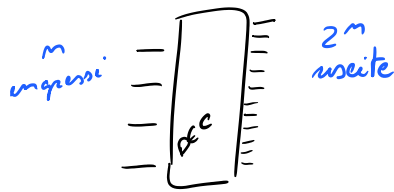
3 entrate
2 bit selezione
(una combinazione non utilizzata)



(log₂ m)

⇒ MUX a 32 entrate: log₂ 32 bit selezione = 5

② DEC :

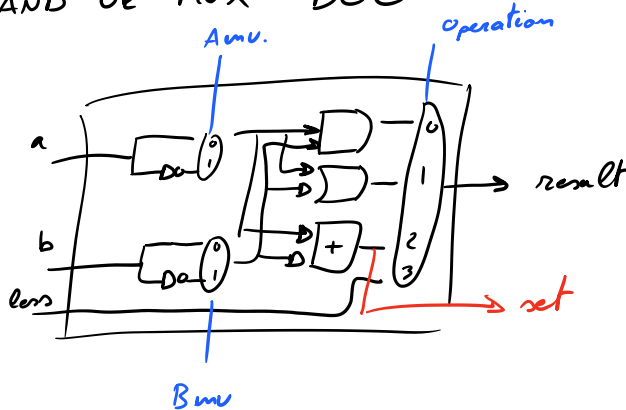


es: input 0000 → output accesso A[0]
→ risp. 3

⇒ DEC con 1024 uscite → $\log_2 1024$ entrate
⇒ 10 entrate

⑬ ALU : somme $A+B$
differenze $A + (\bar{B} + 1)$
op. logiche (AND OR --)

⑭ AND OR MUX DEC



⑮ ROM (Read Only Memory) : scritte durante produzione
(in questo caso non si parla di EEPROM, riscrivibili)

RETI COMBINATORIE (2) :

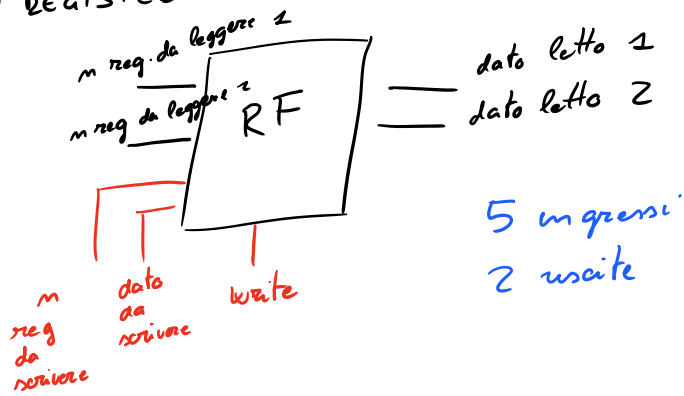
① clock : onda quadra con periodo predeterminato e costante
(periodo = inverso frequenza → $T = \frac{1}{f}$)

$$\Rightarrow T = 0,02 \mu s = 0,02 \cdot 10^{-6} s$$

$$f = \frac{1}{T} = \frac{1}{0,02 \cdot 10^{-6} s} = \frac{1}{2 \cdot 10^{-8} s} = 0,5 \cdot 10^8 = 50 \cdot 10^6 = 50 \text{ MHz}$$

① N.B. : 10^3 Kilo $\sim 2^{10}$ 10^3 Milli.
 10^6 Mega $\sim 2^{20}$ 10^6 Micro
 10^9 Giga $\sim 2^{30}$ 10^9 Nano

④ REGISTER FILE :

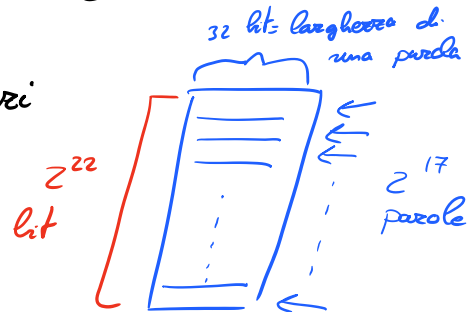


⑤ RAM: 4096 Kbit $\rightarrow 4096 \cdot 2^{10} \text{ bit} = 2^{12} \cdot 2^{10} \text{ bit} = 2^{22} \text{ bit}$
 con parole da 32 bit $\rightarrow$ quante parole posso immagazzinare?

$$\frac{2^{22}}{32} = \frac{2^{22}}{2^5} = 2^{17} \text{ parole}$$

$\Rightarrow$ ho bisogno di 17 bit x indirizzi

$$\text{indirizzi} = \log_2 \text{ parole}$$



⑥ RAM 10 bit

INDIRIZZO da 3 bit $\rightarrow$ parole = $3^2 = 9$

$$\text{dimensione parola} = \frac{\text{dimensione memoria}}{\text{parole}} = \frac{10}{9} = 1,2$$

$\Rightarrow$ una parola ha dimensione 2 (non posso ridurla a 1)

la memoria contiene 5 parole ($\frac{10}{2} = 5$)

3 indirizzi non vengono utilizzati (101, 110, 111)

⑦ indirizzo a 30 bit
 parola a 64 bit

$$2^{30} \text{ parole} \cdot 64 \frac{\text{bit}}{\text{parola}} = 2^{30} \cdot 2^6 = 2^{36} \text{ bit}$$

$$\rightarrow \frac{2^{36} \text{ bit}}{2^3 \frac{\text{bit}}{\text{B}}} = \frac{2^{33} \text{ B}}{2^{20} \frac{\text{B}}{\text{MB}}} = 2^{13} \text{ MB} = 8192 \text{ MB}$$

(8-9) RAM = Random Access Memory =
stesso tempo per accedere a tutte le aree di
memoria

* SRAM più piccola DRAM
(MB) (GB)

* DRAM richiedono refresh

* DRAM più lente SRAM

* il refresh ritarda l'accenso ai dati.

(10)

indirizzo	dato
0x FFFF0000	0x FFFFFFFF
0x - - - - 0020	0x FF - - - - FE
0x - - - - - 40	0x - - - - FD
0x - - - - - 60	0x - - - - FC
0x - - - - - 80	0x - - - - FB

! é in HEX $\Rightarrow$ non ho subito
0x - - - - 100
MA
0x - - - - 0A0

0x - - - - A0	0x - - - - FA
0x - - - - C0	0x - - - - F9
0x - - - - E0	0x - - - - F8
0x - - - - 100	0x - - - - F7

FSM:

esempio: semaforo

esempio: semaforo

OUTPUT: NSlite : quando a 1 accendo NS
EWlite : quando a 1 accendo EW

INPUT: NScar : sensore che rileva macchina NS
EWcar : sensore che rileva macchina EW

STATI: NSgreen : semaforo verde NS
EWgreen : semaforo verde EW

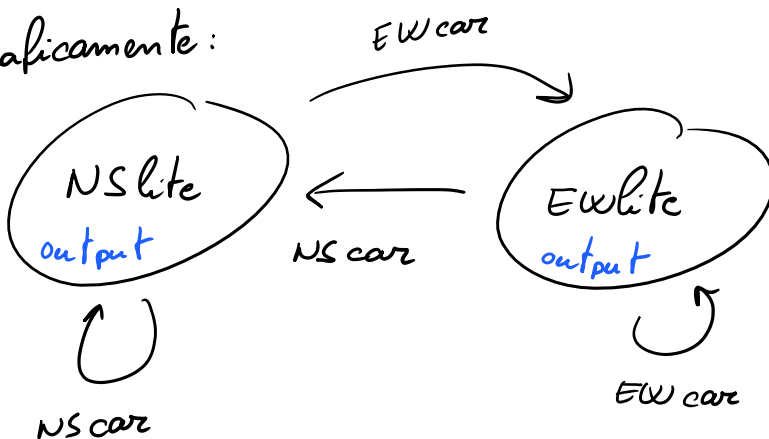
funzione nextstate:

CURRENT STATE		INPUT NSCAR/EWCAR		NEXT STATE
STATO ①	NS green	0	0	NS green
	NS green	0	1	EW green
	NS green	1	0	NS green
	NS green	1	1	EW green
STATO ②	EW green	0	0	EW green
	EW green	0	1	EW green
	EW green	1	0	NS green
	EW green	1	1	NS green

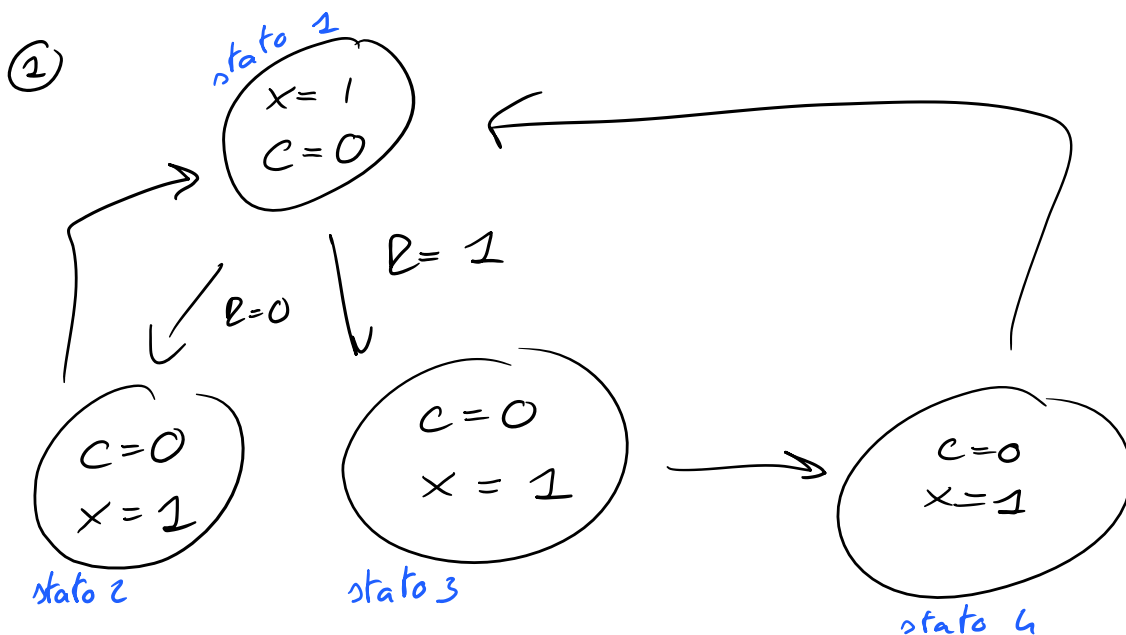
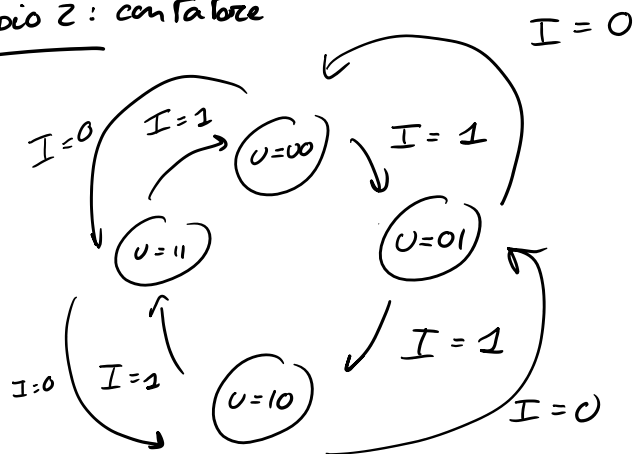
funzione output: (N.B.: nella macchina di Moore!)
l'output dipende solo dallo stato.

CURRENT STATE	OUTPUT NSlite EWlite	
tutte i possibili stati NS green	1	0
EW green	0	1

graficamente:



esempio 2: contatore



funzione nextstate:

CURR. STATE	INPUT R	NEXT STATE
stato 1	0	stato 2
stato 2	1	stato 3
stato 3	0/1	stato 1
stato 4	0/1	stato 1

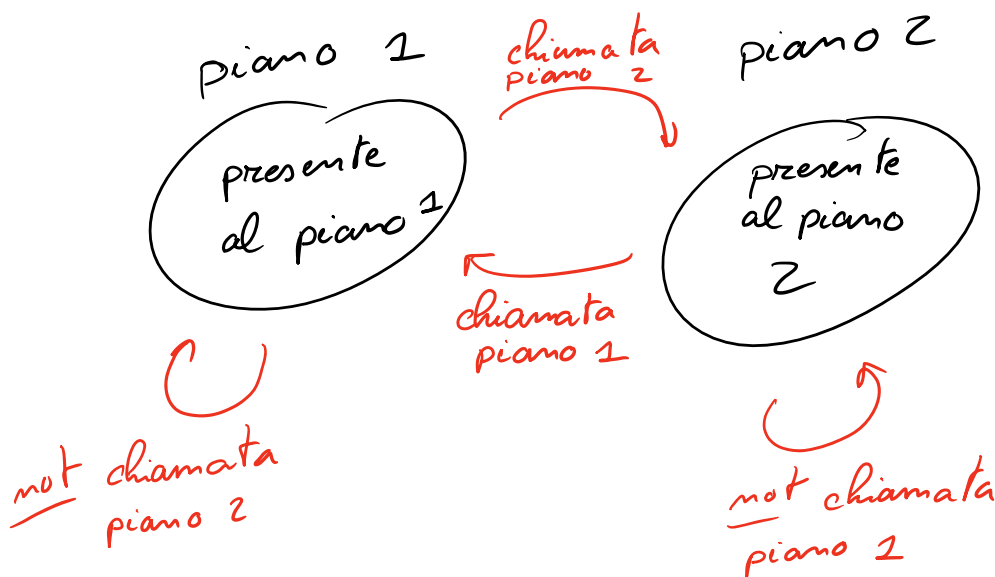
stato 4 0/1 stato 1 ✓

funz. output :

CURR. STATE OUTPUT
 x | c

stato	1	1	0
stato	2	0	1
stato	3	0	1
stato	4	0	1

② ascensore a 2 piani



funz. nextstate :

CURR. STATE INPUT NEXT STATE
 chiamata / chiamata
 piano 1 / piano 2

piano 1	0	0	piano 1
piano 1	0	1	piano 2
piano 1	1	0	piano 1
piano 1	1	1	piano 2

piano 2	0	0	piano 2
piano 2	0	1	piano 2
piano 2	1	0	piano 2
piano 2	1	1	piano 1

funz. output :

CURR. STATE

OUTPUT
 presente / presente
 piano 1 piano 2

piano 1	1	0
piano 2	0	1

① Lab 2 es 4:

a → accenso per : 00 10 11
 b → - - - : 00 01 10 11
 c → : 00 01 11
 d → : 00 10 11
 e → : 00 10
 f → : 00
 g → : 10 11

$$0 \times 3A = 3A_{16}$$

$$\textcircled{1} 19_{16} = \underline{10011}_2 = 0 \times 13 = 1 \cdot 16^1 + 3 \cdot 16^0 = 16 + 3$$

$$128_{16} = \underline{10000000}_2 = 80_{16}$$

$$64738_{16} =$$

$$\textcircled{2} \begin{array}{l} \text{E same} \quad 2008 \quad \text{Hello, world} \\ 4573616065 \quad 32303038 \quad 48656660 \end{array}$$

$$\begin{array}{l} \downarrow \downarrow \quad \downarrow \downarrow \downarrow \downarrow \quad \downarrow \downarrow \quad \downarrow \\ 69 \quad 484856 \quad 72 \quad 33 \end{array}$$

$$\begin{array}{l} 69 \quad 3+ \\ = 112+ \\ \frac{3}{115} \\ 2+ \\ 3 \cdot 16 = \\ 48+2 \\ = 50 \end{array}$$

$$\textcircled{3} -12_{ca2}$$

$$+12 : 00001100 \rightarrow ca_1$$

$$\begin{array}{r} 11110011 + \\ 1 \\ \hline 11110100 \quad ca_2 \\ \underline{F \quad 4} \end{array}$$

$$-34$$

$$+34 = 00100010$$

$$\begin{array}{l} 11011110 \quad ca_2 \\ \underline{13=D \quad 14=E} \end{array}$$

$$\textcircled{4} +128 =$$

$$-1$$

$$+1 = 00000001$$

$$-1 = 11111111_{ca_2}$$

$$\begin{array}{l} X+1 \\ \uparrow \uparrow \uparrow \\ 1204331 \end{array}$$

↑ ↑ ↑
120 43 31

↑
121

⑤

2,625
10.101

27	2	1
13	2	1
6	2	0
3	2	1
1	2	1

FM
10.101 → 1.0101 × 2¹
mantissa exp ecc. 127

0,625 ×
2 =

1,250

0,250 ×
2 =

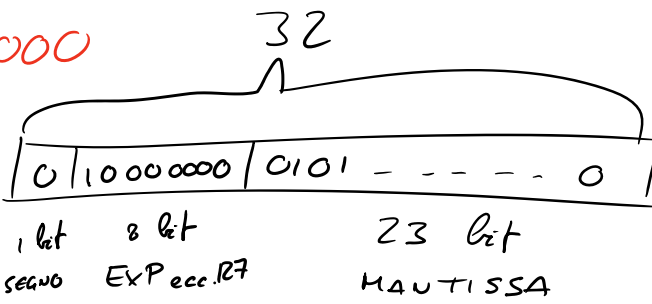
0,500 ×
2 =

1,000

0,000

1 +
127 =
128

10000000
exp



① segno (+)

② exp ecc. 127 = 128

vero exp = 128 - 127 = 1

③ mant = 1.0101 × 2¹ = 10.101
mant.

$\underbrace{\hspace{10em}}$
form. mor.

$$\begin{array}{l} \underbrace{\hspace{1em}} \underbrace{\hspace{1em}} \\ 2 \quad 1 \cdot 2^{-1} = 0,5 \\ \quad 1 \cdot 2^{-3} = 0,125 \end{array}$$

=

$$2,625$$

2,2 TB in
Dischi da 50 GB $\Rightarrow$

$$2,2 \text{ TB} = 2,2 \cdot 1024 \text{ GB} \\ = 2.252,8 \text{ GB}$$

$$2.252,8 : 50 = 45,056 \\ \rightarrow 46$$

Disco da 8 MB $\rightarrow b$

$$64 \cdot 1024 \cdot 1024 \\ = 65536$$

Disco da 122 MB $\rightarrow B$

$$122 \cdot 1024 \cdot 1024 \\ = 127926272$$

361 KB $\rightarrow B$

$$361 \cdot 1024 = \\ 369664$$

8 lit
fra 14,4 e 40 } voglio rappresentare
30,913
40 - 14,4 = 25,6
100010

11001, 111

$$25,6 : 256 = 0,1$$

$$30,913$$

0,13

. data 01 0 00 0 78

6 word 2

1 word 20

1 word 37

a word 35

0 word 17

$$\begin{array}{r} 18 + \\ 16 = \\ \hline 2E \end{array}$$

Esercitazione 5

venerdì 5 maggio 2017 10:49

→ CONTROL UNIT: fsm (implementata con porte logiche)

È ogni volta che voglio aggiungere qualche istruzione dovrò modificare C.U.

→ se fanno "operazioni inutili"/ridondanti; posso specificarlo comunque nel diagramma delle FSM

→ FIG. 5:30 = istruzioni comuni a tutte le operazioni (ho 5 cicli di clock per eseguire l'istruzione; la nostra implementazione è multiciclo)

(L) alcune istruzioni sono più brevi di 5 cicli; i colpi di clock inutili NON sono sprecati ma fanno la fetch dell'istruzione successiva.

→ schema multiciclo, lo capisco da:

- CU (@ unità sequenziale per ciclo singolo)

- IR + MDR

(tra un colpo di clock e un altro devo scambiare informazioni)

↑ aggiornati quando il collegamento alla CU va a 1

ES 1: tutte le istr. usano la ALU!
(vedi fase 1 tabella 5.30)

- salti assoluti NON usano ALU (JR)

- ADD, SUB sì (ovvio)

- SW, LW sì (calcolo offset)

- JEQ sì (zero)

=> resp: false

ES 2:

PC → indirizzo istruzione.

IR → istruzione vera e propria

ES 3: quali componenti usa CI?

pseudoistruzione: CI (piccolo) → ADDI

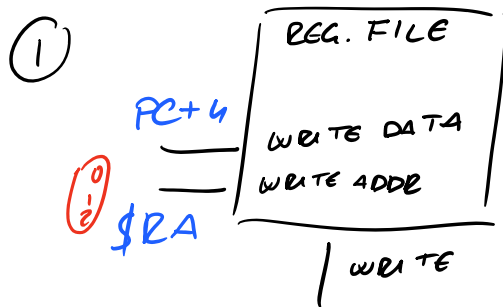
- REGISTER FILE, ALU (ovvio)
- NO memory (faccio solo ADDI)

ES 4: aggiungere JAL

→ dovrò modificare sia l'automa (CU) + collegamenti

→ cosa fa la JAL?

- ① lettura 26 bit indirizzo: già presente
+ sign extension: già presente



modifica: ① tutto ingresso
multiplexer write
addr

② il segnale di
controllo può
valere 2
(mi servono 2 fili.)

② sign extension già presente

② sign extension già presente



devo cambiare !!!
FSM ...

ESECUZIONE I/O

venerdì 19 maggio 2017 10:50

→ INTERRUPT: • la periferica segnala (elettricamente sul bus controllo) che ha ricevuto un input / ha emesso un output; a quel punto la normale esecuzione delle istruzioni

→ nello STATUS REGISTER ogni bit corrisponde ad una periferica: se a 0 $\Rightarrow$ interrupt disabilitato
se a 1 $\Rightarrow$ interrupt abilitato

→ reg CAUSE modificato $\oplus$ spesso perché registra i singoli eventi di interrupt

→ il SO legge CAUSE e scrive STATUS

→ SPIM ha: • RECEIVER $\leftarrow$ singolo carattere in ingresso
(come periferiche I/O) • TRANSMITTER $\leftarrow$ " " in uscita

sono separati e indipendenti.

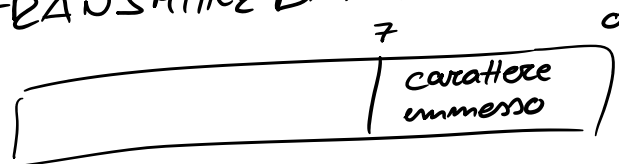
→ conf. memory mapped:

REC. CTRL 0xFFFF0000
REC. DATA 0xFFFF0004

TRANSMITTER CTRL 0x...08

TRANSMITTER CTRL 0x...0c

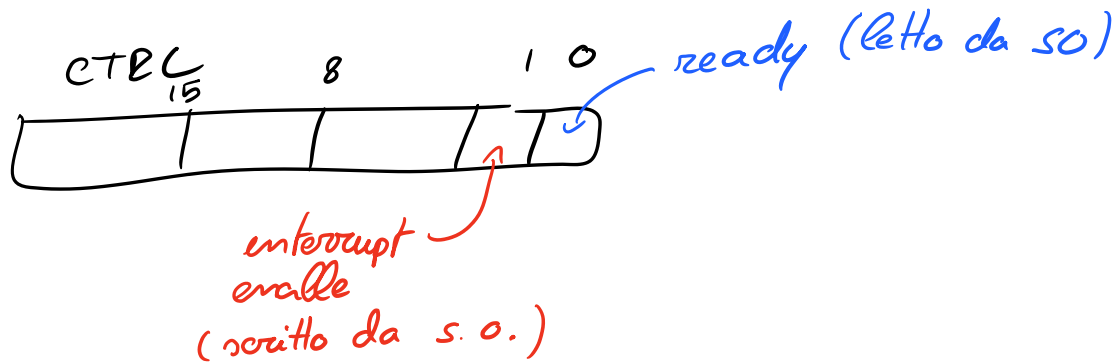
→ TRANSMITTER DATA:



farò LW 0xFFFF000c

01 1. DEADY: periferica pronta a ricevere /

→ Bit di READY: periferica pronta a ricevere /
(in CTRL) mostrare dato?



↑ impostando questo a 0 posso
simulare POLLING / BUSY WAIT
(ciclo infinito per vedere se è arrivato
qualcosa)

BusyWait Read: lw \$t1, 0(\$to)
andi \$t1, \$t1, 0x1
beqz \$t1, BusyWait Read

LAB DATAPATH

domenica 21 maggio 2017 18:50

ES. 1

① FETCH: $IorD \rightarrow 0$
[$ALUSrcA \rightarrow 0$
 $ALUSrcB \rightarrow 1$
 $ALUOut$ contiene $PC+4$
 $PCSrc \rightarrow 0$
 $PCWriteCond \rightarrow 0$
 $PCWrite \rightarrow 1$
 PC contiene $PC+4$

② DECODE:

$RR1 \rightarrow ist[25-21]$
 $RR2 \rightarrow ist[20-16]$
 $RDst \rightarrow 1$
 $WR \rightarrow ist[15-11]$
 $RegWrite \rightarrow 0$

"mutile 1"
 $A \leftarrow B$ [$A \rightarrow RR1$
 $B \rightarrow RR2$

"mutile 2"
calcolo
branch
addr [calcolo sign extend, shift left 2 (uscita 2-3 mux)
 $ALUSrcB \rightarrow 3$
 $ALUSrcA \rightarrow 0$
 $ALUOut \rightarrow risultato$

③ EXECUTE:

$ALUOut \rightarrow A+B$
 $MemToReg \rightarrow 0$

ESERCITAZIONE - CACHE

venerdì 26 maggio 2017 11:55

ES. 1:

In quanto si esegue questo programma?

- senza cache 200 ms (accesso RAM 5 ms)
- cache 2 ms
- hitrate 0,9 → 9/10 hit → accessi cache
1/10 miss → accessi ram

$$\begin{array}{lcl} \text{tempo totale} & 200 \text{ ms} & \\ \text{tempo accesso} & 50 \text{ ns} & \Rightarrow \frac{200 \cdot 10^{-3}}{50 \cdot 10^{-9}} \Rightarrow 4 \cdot 10^6 \text{ accessi in memoria} \end{array}$$

$$\underbrace{4 \cdot 10^6 \cdot 0,9 \cdot 2 \text{ ms}}_{\text{hit}} + \underbrace{4 \cdot 10^6 \cdot 0,1 \cdot 50 \text{ ms}}_{\text{miss}}$$

ES 2:

- 5 cicli per hit
- 15 miss penalty
- 0,75 hit probability

$$\underbrace{5 \cdot 0,75}_{\text{hit}} + \underbrace{15 \cdot 0,25}_{\text{miss}}$$

DMA

ES 1:

Costo del trasferimento in 2 casi:

- testo da $2 \cdot 10^6$ byte
- CPU 500 MHz
- 1 KB/s (1000 bit/s)
- impostazione DMA 10 cicli clock

nei casi: $\left\{ \begin{array}{l} \text{CONTROLLO PROGRAMMA} \\ \text{tempo esecuzione} = \text{byte da trasferire} / \text{byte secondo} \\ \text{cicli} = \text{tempo} \cdot \text{frequenza} \\ \text{DMA} \\ \rightarrow \text{solo tempo setup} \\ 10 \cdot \text{tempo singolo ciclo} \end{array} \right.$

DATA PATH

- aggiunta JR $\textcircled{!}$ il secondo registro conterrà 0
- aggiunta JAL $\Rightarrow$ aggiungo multiplexer con la possibilità di mettere su reg. write il reg 31
- $\textcircled{+}$ multiplexer per data write con valore PC

→ CAUSE: 0x 0000 0024

- - - 0000 0 [010 0100]

1+8=9

Bp = Break Point

10

0 [001 00] 00 ← AdEEL
u

→ ES 3:

\$k0 ← CAUSE

sm/ → 00 - - 101 - - [00100]
exc. code

andi 00 - - 000 - [11111]

2

0 - - 000 - 0 [00100]

per avere
un interrupt
deve essere = 0

→ ES 4:

cause = 14

0 [001 0100]

5 = AdEs

BADVADDR = 0x1001000~~0~~
1

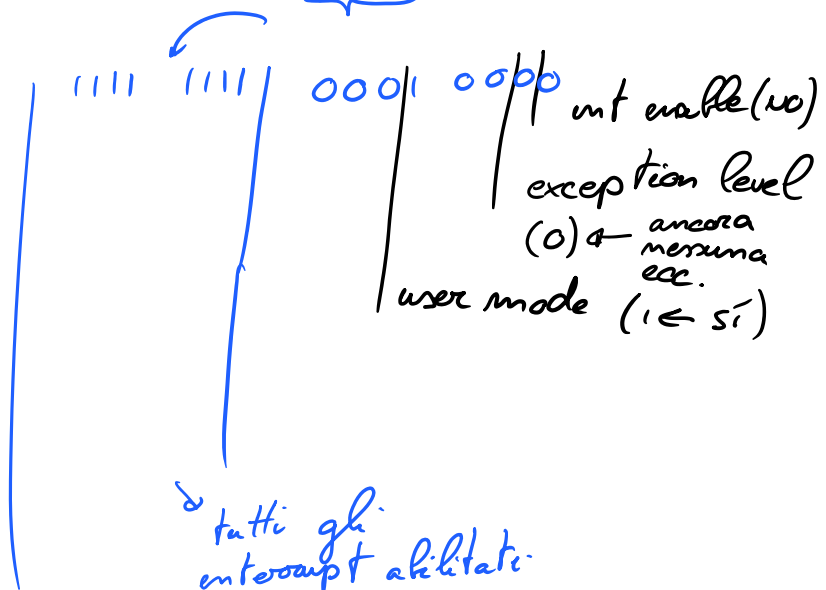
EPC =

0x00400024
8
C
30
34
38

→ BREAK:

CAUSE: 00000024
0010 0100
9

→ STATUS: 3000FF10



Esercizi Cache

① 16 Blocchi
4 word/Blocco

=> a che Blocco viene salvato l'indirizzo 128 (indirizzamento al byte)

idea: trovare il Blocco "corretto"

$$\frac{\text{ind. dato (byte)}}{\text{byte per Blocco}}$$

$$\% \text{ n. Blocchi}$$

$$\frac{128}{16} \% 16$$

$$= 8 \% 16 = 8$$

idea: posso vedere la cache come un array circolare!

② 16 Blocchi

$$4 \text{ word/Blocco} \rightarrow 4 \text{ word} \times \frac{4 \text{ byte}}{\text{word}} = 16 \text{ byte}$$

=> indirizzo 64

$$\frac{\text{ind. dato (byte)}}{\text{byte per Blocco}}$$

$$\% \text{ n. Blocchi}$$

$$\frac{64}{16} \% 16 \Rightarrow 4 \% 16 = 4$$

$$\frac{64}{16} \% 16 \Rightarrow 4 \% 16 = 4$$

③ 16 Blocchi

$$8 \text{ word/Blocco} \rightarrow 8 \times 4 = 32 \text{ byte/Blocco}$$

=> ind. 128

$$\frac{128}{32} \% 16 \Rightarrow 4 \% 16 = 4$$

④ Cache 8 Blocchi di 2 word. Dove quali indirizzi in decimale si mappano sul quarto Blocco.

w1	w5	w5	w7	w9	w11	w13	w15
01	8 9	16 17	24 25	32 33	40 41	48 49	56 57
03	10 11	18 19	26 27	34 35	42 43	50 51	58 59
w2	w6	w6	w8	w10	w12	w14	w16
12 13	20 21	28 29	36 37	44 45	52 53	60 61	68 69
w4	w8	w8	w10	w12	w14	w16	w18
14 15	22 23	30 31	38 39	46 47	54 55	62 63	70 71

1 word → 4 byte

4 Blocco → 8 × 3 = 24

I° indirizzo: 24 fine primo "array"

I° indirizzo: 24 *fine primo "quarto"*

II° indirizzo: $8 \times 8 + 8 \times 3 = 64 + 24 = 88$

III° indirizzo: $8 \times 8 + 8 \times 8 + 8 \times 3 = 128 + 24 = 152$

- ① 5 cicli $\rightarrow$ hit
15 cicli $\rightarrow$ miss penalty
0,75 hit prob.

Quanti cicli servono mediamente $\times$ accedere in mem.?

$$5 \times 0,75 + 15 \times 0,25 = 3,75 + 3,75$$

$$5 \times \frac{3}{4} + 15 \cdot \frac{1}{4} = 7,5$$

① 16 kB dati $\rightarrow 16 \cdot 2^{10} B = 2^4 \cdot 2^{10} B = 2^{14} B \cdot 2^3 \frac{B}{B} = 2^{17}$

• blocchi da 4 word $\rightarrow 4 \times 4 = 16 B \rightarrow 16 \times 8 = 128 B$

• indirizzi a 32 bit $\hookrightarrow 2^7 B$

a) n° blocchi = $\frac{16 \cdot 2^{10}}{2^4} \rightarrow 2^{10}$ blocchi.

b) tag = $32 - (10 + 7 + 2 + 2) = 32 - (21) = 11$

c) bit = $2^{17} \cdot (11 + 1 + 4 \cdot 32) = [\dots]$

① 4 blocchi da 2 word

• loop eseguito 10 volte su 0, 1, 2, 3, 4 quanti miss?

w1	w3	w5	w7
01 23			

w1 0 1 2 3	w3	w5	w7
w2 4	w4	w6	w8

$\Rightarrow$ risp:

①

Quanti mesi ci saranno con i seguenti accessi?

① 0 0 ② 8 ③ 16 0 ④ 4 ⑤ 32 8 ⑤

0 1 2 3	4 5 6 7	8 9 10 11	12 13 14 15
------------------	------------------	--------------------	----------------------